CredShields

Smart Contract Audit

February 2, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Macroscape for their UR Wife project between January 21, 2026, and January 22, 2026. A retest was performed on January 30, 2026.

Author
Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers
Aditya Dixit (Research Team Lead), Shreyas Koli(Auditor), Naman Jain (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for
UR Wife

Table of Contents

1. Executive Summary ----------

UR Wife engaged CredShields to perform a smart contract audit from January 21, 2026, to January 22, 2026. During this timeframe, 5 vulnerabilities were identified. **A retest was performed on January 30, 2026, and all the bugs have been addressed.**

During the audit, 0 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "UR Wife" and should be prioritized for remediation, and fortunately, none were found.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Smart Contract	0	0	0	3	2	0	**5**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in the Smart Contract's scope during the testing window while abiding by the policies set forth by UR Wife's team.

State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both UR Wife's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at UR Wife can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, UR Wife can future-proof its security posture and protect its assets.

2. The Methodology ----------

UR Wife engaged CredShields to perform a Smart Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from January 21, 2026, to January 22, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
Audited Scope: https://github.com/UR-Wife/claim-ur-wife-contracts/tree/86a3342ec11f2aa9281e20bb209426cb72b61c51 **Retested Scope:** https://github.com/UR-Wife/claim-ur-wife-contracts/tree/da22621914fd67041975d278133214938e25e2bf

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

UR Wife is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. Informational

We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. Low

Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 5 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Malicious User Can Block Token Swaps Opponents	Low	Denial of Service (SCWE-087)	Fixed
Missing _disableInitialzers() Call in Constructor	Low	Insecure Upgradeable Proxy Design (SCWE-005)	Fixed
Floating and Outdated Pragma	Low	Floating Pragma (SCWE-060)	Fixed
Redundant Base URI Update Allowed Due to Missing Equality Check	Informational	Lack of Input Validation (SC04-Lack Of Input Validation)	Fixed
Operator Can Arbitrarily Manipulate User Balances	Informational	Centralization Risk	Won't Fix

Table: Findings and Remediations

5. Bug Reports ----------

Bug ID #L001 [Fixed]

Malicious User Can Block Token Swaps Opponents

Vulnerability Type
Denial of Service (SCWE-087)

Severity
Low

Description
The URWIFE_ERC1155.directSwap function is designed to allow an authorized Operator to atomically swap one ERC1155 token between two users, enabling controlled gameplay interactions. The function performs balance checks and then transfers one token from each user to the other using _safeTransferFrom, relying on the ERC1155 standard transfer flow. However, _safeTransferFrom invokes the onERC1155Received hook on the recipient if the recipient is a contract. If one of the participating users is a malicious contract that intentionally reverts within onERC1155Received, the entire directSwap transaction reverts. This allows the malicious user to unilaterally prevent swaps involving their address, with the root cause being the use of safe transfers to arbitrary recipients without handling malicious receiver behavior.

Affected Code
- https://github.com/UR-Wife/claim-ur-wife-contracts/blob/86a3342ec11f2aa9281e20bb209426cb72b61c51/contracts/URWIFE_ERC1155.sol#L303C5-L339C6

Impacts
A malicious user deploys a malicious contract that always reverts in onERC1155Received and uses it as his player address. Whenever a genuine user is matched against a malicious user, the Operator's call to URWIFE_ERC1155.directSwap always reverts, preventing genuine user from completing the swap and effectively blocking gameplay against a malicious user.

Remediation
It is recommended to prevent griefing by disallowing users with contracts when onboarding.

Retest

This issue has been fixed by adding checks to avoid the directSwap between the user as a contract.

Bug ID #L002 [Fixed]

Missing _disableInitialzers() Call in Constructor

Vulnerability Type
Insecure Upgradeable Proxy Design (SCWE-005)

Severity
Low

Description
When using UUPSUpgradeable, the best practice is to call _disableInitizliers() in the constructor since the initializer function in the implementation contract can be called by anyone. The proxy's storage is separate from the implementation contract's storage. Directly initializing the implementation means you're not affecting the state that will be used when interacting via the proxy. This can lead to confusion or unexpected behavior if someone accidentally initializes the implementation contract.

Affected Code
- https://github.com/UR-Wife/claim-ur-wife-contracts/blob/86a3342ec11f2aa9281e20bb209426cb72b61c51/contracts/URWIFE_ERC1155.sol#L54

Impacts
An uninitialized implementation contract can be taken over by an attacker.

Remediation
It is recommended to call _disableInitializers() in the constructor.

Eg:

```
constructor(){
  _disableInitializers();
}
```

Retest
This issue has been fixed.

Bug ID #L003 [Fixed]

Floating and Outdated Pragma

Vulnerability Type
Floating Pragma (SCWE-060)

Severity
Low

Description
Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.
The contract allowed floating or unlocked pragma to be used, i.e., >= 0.8.24. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

Affected Code

- https://github.com/UR-Wife/claim-ur-wife-contracts/blob/86a3342ec11f2aa9281e20bb209426cb72b61c51/contracts/URWIFE_ERC1155.sol#L3

Impacts
If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.
Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.
The likelihood of exploitation is low.

Remediation
Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.32 pragma version
Reference: https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-060/

Retest
This issue has been fixed.

Bug ID #I001 [Fixed]

Redundant Base URI Update Allowed Due to Missing Equality Check

Vulnerability Type
Lack of Input Validation (SC04-Lack Of Input Validation)

Severity
Informational

Description
The `setBaseURI` function allows an account with `ADMIN_ROLE` to update the base URI used for token metadata, and it is expected to be called only when a meaningful metadata change is required. The function retrieves the current URI using `super.uri(0)`, updates the base URI via `_setURI(newBaseURI)`, and emits a `BaseURIUpdated` event to signal the change to off-chain indexers and users.

However, the function does not verify that the provided `newBaseURI` differs from the existing base URI. As a result, the administrator can set the same URI value again, which unnecessarily triggers state changes and emits a misleading update event despite no effective modification. The root cause is the absence of an equality check between the old and new base URI values before performing the update.

Affected Code
- https://github.com/UR-Wife/claim-ur-wife-contracts/blob/86a3342ec11f2aa9281e20bb209426cb72b61c51/contracts/URWIFE_ERC1155.sol#L398C5-L407C6

Impacts
Off-chain indexers, marketplaces, and monitoring systems may process redundant update events, leading to unnecessary re-indexing, increased operational overhead, and potential confusion for token holders who interpret the event as a real metadata change.

Remediation
It is recommended to add a check to ensure the new base URI is different from the existing base URI before updating and emitting the event.

Retest
This issue has been fixed by adding required check.

Bug ID #I002 [Won't Fix]

Operator Can Arbitrarily Manipulate User Balances

Vulnerability Type
Centralization Risk

Severity
Informational

Description
The contract implements a fully custodial ERC1155 token model where users hold balances that represent in-game assets, but are unable to transfer, approve, or otherwise control those assets themselves. All lifecycle actions such as minting, burning, and swapping are intended to be performed by a trusted operator address with `OPERATOR_ROLE`, while administrators can pause the system and modify metadata, and upgraders can change the implementation.

However, the `OPERATOR_ROLE` is granted broad and unilateral authority to call `mintTo`, `mintBatch`, `burnFrom`, `burnBatch`, and `directSwap` for any user address without user consent or on-chain verification of off-chain game state. There are no protocol-level constraints, timelocks, multi-signature requirements, or invariant checks limiting how or when these powers can be exercised, making correct behavior entirely dependent on off-chain trust assumptions.

Affected Code
- https://github.com/UR-Wife/claim-ur-wife-contracts/blob/86a3342ec11f2aa9281e20bb209426cb72b61c51/contracts/URWIFE_ERC1155.sol#L132
- https://github.com/UR-Wife/claim-ur-wife-contracts/blob/86a3342ec11f2aa9281e20bb209426cb72b61c51/contracts/URWIFE_ERC1155.sol#L163
- https://github.com/UR-Wife/claim-ur-wife-contracts/blob/86a3342ec11f2aa9281e20bb209426cb72b61c51/contracts/URWIFE_ERC1155.sol#L308

Impacts
If the operator key is compromised or behaves maliciously, users can have tokens arbitrarily minted to dilute supply, burned to erase balances, or swapped between accounts without approval, resulting in loss of assets, loss of game progress, and complete reliance on operator integrity for fairness and safety.

Remediation
It is recommended to document this trust model explicitly.

Retest
Acknowledged

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com